Apunte de Cátedra – Diagrama de Secuencias (UML) en la etapa de Diseño de Sistemas

Objetivo del apunte

Al finalizar la lectura, el/la estudiante debería poder:

- Comprender el sentido del diagrama de secuencias dentro de la etapa de diseño.
- Construir diagramas de secuencia a partir de la descripción textual de un caso de uso.
- Usar el diagrama para asignar responsabilidades (quién hace qué) y descubrir métodos/operaciones.
- Modelar alternativas, condiciones, bucles y errores usando fragmentos combinados (alt/opt/loop/par).
- Evitar errores frecuentes y justificar el diseño elegido.

1) ¿Qué es un diagrama de secuencias y cuál es su sentido en Diseño?

Un diagrama de secuencias muestra la interacción de un conjunto de objetos en una aplicación a través del tiempo y se modela para cada caso de uso. Atender distintos escenarios cada uno con su DS.

Mientras que el diagrama de casos de uso permite modelar una vista de negocios del escenario, el diagrama de secuencia agrega detalle de implementación: objetos/clases involucrados y mensajes intercambiados.

Sentido en diseño: transformar requisitos (casos de uso) en colaboraciones concretas entre componentes/objetos, definiendo responsabilidades, operaciones, interfaces y puntos de integración antes de programar.

¿Qué problema resuelve?

- Baja del “qué” (requisito) al “cómo” (mecánica interna del sistema).
- Hace explícitas las decisiones de diseño: quién valida, quién decide, quién persiste.
- Descubre métodos/operaciones que luego deben existir en clases/servicios.
- Expone alternativas y errores que el diseño debe contemplar (validaciones, rechazos, timeouts).
- Reduce ambigüedad: dos personas pueden implementar módulos distintos con acuerdos claros.

2) Tipos de diagrama: de instancia vs. genérico

De instancia: describe un escenario específico (un escenario es una instancia de la ejecución de un caso de uso).

Genérico: describe la interacción para un caso de uso; utiliza alternativas, condiciones y bucles.

Recomendación didáctica: usar el diagrama genérico como "modelo principal" y diagramas de instancia para ilustrar recorridos puntuales (por ejemplo, un error específico).

3) ¿Qué "produce" un buen diagrama de secuencias?

3.1 Descubrimiento de métodos y contratos

Los mensajes del diagrama suelen convertirse en operaciones concretas en el diseño: métodos de clases del dominio, operaciones de servicios de aplicación, llamadas a repositorios/DAO, endpoints o contratos de integración.

Ejemplo: si aparece el mensaje "validarCupo()", entonces alguna clase/servicio deberá ofrecer esa operación con su firma/efecto correspondiente.

Contrato vs. Interfaz (distinción clave para tu apunte)

Interfaz (lo "sintáctico")
Es el "cómo me llamás": nombre de la operación, parámetros de entrada, valor de retorno errores/excepciones o códigos de error esperables.

Contrato (lo "semántico" + lo no funcional)

Es el "qué prometo si me llamás bien" y "qué te exijo para poder cumplir": precondiciones (qué debe ser verdad antes de llamar), postcondiciones (qué garantizo al finalizar), invariantes (qué propiedades siempre se mantienen), efectos laterales (qué cambia: BD, estado, envío de eventos), modelo de errores (qué errores se reportan y cómo), opcionalmente: protocolo (orden válido de llamadas) y niveles de servicio (latencia, disponibilidad, etc.)

3.2 Asignación de responsabilidades

El diagrama ayuda a ubicar responsabilidades en componentes adecuados, evitando:

- Interfaz/Boundary con lógica de negocio (acoplamiento y baja testeabilidad).
- Un único controlador "Dios" que hace todo.
- Entidades de dominio anémicas (sin comportamiento).

3.3 Identificación de dependencias e integraciones

Al diagramar surgen dependencias reales: persistencia, autenticación/autorización, servicios externos, colas/eventos, etc.

4) Cómo se lee un diagrama de secuencias

El tiempo se representa en dirección vertical: inicia arriba y avanza hacia abajo. Un mensaje más arriba ocurre antes que uno más abajo.

El diagrama tiene dos dimensiones: horizontal (disposición de participantes) y vertical (paso del tiempo).

5) Elementos esenciales

5.1 Líneas de vida (Lifelines)

Representan la existencia de un objeto/participante a lo largo de un período de tiempo.

Participantes típicos en diseño de aplicaciones:

- Actor (persona o sistema externo que inicia).
- «boundary» (pantalla / controlador web / endpoint API).
- «control» (orquestador del caso de uso / servicio de aplicación).
- «entity» (objetos del dominio con reglas del negocio).
- Repositorio/DAO (persistencia).

5.2 Foco de control (Activation)

Rectángulo delgado que representa el período durante el cual el objeto ejecuta una acción. La parte superior se alinea con el comienzo y la inferior con su terminación.

6) Mensajes (sincrónicos, asincrónicos y retornos)

Existen dos tipos principales: sincrónicos y asincrónicos.

- Sincrónicos: se corresponden con llamadas a métodos del objeto receptor. El emisor queda bloqueado hasta que termina la llamada. Se suelen representar con flecha de cabeza llena.
- Asincrónicos: terminan inmediatamente y disparan ejecución independiente (nuevo hilo) o procesamiento desacoplado. Se suelen representar con flecha de cabeza abierta.

También puede representarse la respuesta/retorno como flecha discontinua. Buena práctica: dibujar retornos solo cuando aportan información relevante (id generado, error, token, etc.).

Mensajes útiles adicionales (recomendado)

- Self-message: un objeto se envía un mensaje a sí mismo (cálculos/validaciones internas).
- Create: creación de una instancia durante la secuencia.
- Destroy: destrucción de una instancia (menos frecuente en sistemas de negocio).
- Found/Lost: mensajes cuyo origen/destino no se modela (integraciones externas parciales).

7) Fragmentos combinados: alternativas, opciones y bucles

Los fragmentos combinados permiten expresar lógica de control sin perder el enfoque orientado a objetos. Son clave para que el diagrama sea verdaderamente "de diseño".

- alt: alternativas mutuamente excluyentes (if/else). Cada rama debe tener guarda (condición).
- opt: fragmento opcional (if sin else).
- loop: repetición (for/while; validaciones por lista, reintentos).
- par: paralelismo/concurrencia (acciones simultáneas).

Guardas (guards): condiciones explícitas en corchetes, por ejemplo [cupoDisponible], [pagoRechazado].

8) Nivel de detalle según fase y audiencia

El detalle del diagrama depende de la fase del proyecto, lo que se pretende comunicar y a quién. En fases iniciales pueden usarse participantes "macro" (módulos, paquetes, subsistemas). En fases avanzadas se llega al nivel de clases reales, métodos y parámetros para acordar implementación entre programadores.

Guía práctica de niveles

- Nivel 1 – Macro-diseño: lifelines por módulo/capa; mensajes de alto nivel; objetivo: dependencias y orden global.

- Nivel 2 – Diseño OO (recomendado en la cursada): BCE o capas (Controller/Service/Domain/Repo); verbos del dominio; alt/opt/loop; objetivo: responsabilidades y métodos.

- Nivel 3 – Diseño contractual: firmas con parámetros, tipos de retorno, errores/excepciones, efectos laterales; objetivo: implementar sin ambigüedad.

9) Metodología para construirlo a partir de un caso de uso

1. Seleccionar el escenario: comenzar por el escenario principal (happy path) y luego modelar 1–2 escenarios alternativos críticos.
2. Identificar participantes mínimos: Actor + «boundary» + «control» + entidades relevantes + repositorio/integración.
3. Recorrer los pasos del caso de uso ("caminar" el texto): cada paso se traduce en uno o más mensajes.
4. Introducir fragmentos combinados: decisiones → alt/opt, repeticiones → loop, concurrencia → par.
5. Asignar responsabilidades: UI no decide negocio; «control» orquesta; dominio aplica reglas; repositorio persiste.

6. Revisar coherencia: mensajes sin dueño (faltan métodos/clases), UI con lógica (reubicar), controlador dios (dividir).

10) Corrección formal: Arquitectura en 3 capas vs. MVC vs. MVP

En algunos textos se mezclan “3 capas” con MVC/MVP. Formalmente son conceptos distintos y conviene diferenciarlos:

10.1 Arquitectura en 3 capas (Presentación – Negocio – Datos)

- Es un estilo arquitectónico que separa responsabilidades por capas: presentación (UI), lógica de negocio (servicios/dominio) y acceso a datos (repositorios/DAO).
- Su objetivo principal es reducir acoplamiento, mejorar mantenibilidad y habilitar desarrollo y pruebas por separado.

10.2 MVC (Model–View–Controller)

- Es un patrón de diseño para separar la interfaz de usuario de la lógica: la View muestra/recoge interacción; el Controller interpreta acciones del usuario y coordina; el Model representa el estado y reglas del dominio (o parte de ellas).
- En muchos frameworks web modernos, “Controller” termina siendo un punto de entrada (endpoint) y parte de la orquestación se delega a servicios.

10.3 MVP (Model–View–Presenter)

- Es un patrón de presentación donde la View es más “pasiva”: delega en el Presenter la coordinación de la lógica de presentación.
- El Presenter realiza el “binding” de datos y decide qué se muestra; el Model sigue representando el dominio/datos.

10.4 Recomendación para diagramas de secuencia en la materia

Para diagramas de diseño de aplicaciones, una práctica clara es estereotipar los participantes así:

- «boundary»: pantalla / endpoint / controlador web (capa de presentación).
- «control»: servicio de aplicación o controlador del caso de uso (orquesta el escenario).
- «entity»: entidades de dominio (reglas del negocio).
- Repositorio/DAO: acceso a datos (capa de datos).

11) Errores típicos (y por qué perjudican el diseño)

- UI/«boundary» con lógica de negocio: aumenta acoplamiento y dificulta pruebas.
- Controlador “Dios”: concentra demasiadas responsabilidades y vuelve frágil el diseño.
- No modelar alternativas (alt/opt): oculta validaciones, excepciones y el mundo real.
- Mensajes sin semántica (process/doIt): no ayudan a descubrir operaciones del dominio.

- Bajar demasiado el nivel (SQL en flechas): pierde abstracción de diseño OO.
- No representar persistencia/integración: genera sorpresas en implementación.

12) Checklist de calidad (rúbrica rápida)

Un diagrama de secuencias está “bien” si:

- Incluye participantes correctos (actor + capas/roles).
- Usa mensajes con verbos del dominio y orden lógico.
- Modela alternativas y errores cuando corresponde (alt/opt).
- Incluye persistencia/integraciones relevantes.
- No delega reglas de negocio en la UI.
- Permite derivar métodos/clases a partir de los mensajes.

13) Ejemplo completo (operativo para la cursada)

Caso: “Registrar inscripción a evento” (principal + alternativo sin cupo)

Este ejemplo muestra separación de roles (boundary/control/entity/repositorio), decisión explícita con alt y descubrimiento de métodos como hayCupo(), obtenerEvento() y guardar().

```
@startuml
actor Usuario
boundary "PantallaInscripcion" as UI
control "InscripcionController" as C
control "InscripcionService" as S
entity "Evento" as E
entity "Inscripcion" as I
database "InscripcionRepo" as R

Usuario -> UI : completarFormulario(datos)
UI -> C : enviarSolicitud(datos)

C -> S : registrarInscripcion(datos)
S -> R : obtenerEvento(eventoId)
R --> S : E

S -> E : hayCupo?()

alt [cupoDisponible]
  S -> I : crear(datos, E)
  S -> R : guardar(I)
  R --> S : idInscripcion
  S --> C : confirmacion(idInscripcion)
  C --> UI : mostrarConfirmacion(idInscripcion)
else [sinCupo]
  S --> C : error("Sin cupo")
```

```
  C --> UI : mostrarError("Sin cupo")
end
@enduml
```

Anexo: Nota sobre figuras del apunte original

El apunte original incluía referencias a imágenes/figuras. En esta versión se conserva el contenido textual y se incorporan ejemplos reproducibles en PlantUML para facilitar su uso en clase y en trabajos prácticos.